Dojo | Hints and explained solutions

Spoilers begin after this page. Use the concept guide and cheatsheet first, then reveal help only for the current exercise. The problems themselves are in the separate workbook.

Read one layer at a time

Each entry contains Hint 1, Hint 2, a reasoning paragraph, reference code and costs. More than one implementation may be correct. Compare the invariant and contract, not cosmetic choices. A solution that uses fewer lines can still be worse if it loses clarity or violates a required representation.

The code targets Source Academy Python §3. Use its predeclared pair/head/tail/stream primitives. memo_fun is defined explicitly below when needed. Respect additional restrictions in each assessment. These are worked reference solutions, not the only valid approaches.

Cost conventions

n and m are input lengths, R and C grid dimensions, t token count, and N a total expanded output size where stated. Output space is distinguished from auxiliary state and recursive stack. Python §3 supports proper tail calls; retained closures and non-tail pending work still consume space. Numeric arithmetic is treated as constant-time unless the notes explicitly discuss large integers.

Structural constraints

Destructive exercises require original node identities, unchanged heads and proper terminating chains. Temporary search mutation must be fully undone. Stream examples assume an explicit memo_fun wrapper where required; stream_tail forces a thunk; caching is explicit in these definitions.

Reference files

The website provides D01-D36 reference programs as separate downloads. Each includes its needed support and test helpers. Starter programs contain no reference answer. G06, G09-G10, G12, G14, G18 and G20 explain the Practice A patterns. Use print(...) to display an expression result in the playground.

All 36 runnable examples have been checked against a conservative §3 syntax audit and supplemental logic tests. Those offline logic checks do not certify the Source Academy runtime. The provided §3 programs let you run the checks in that runtime directly. Passing finite tests is not proof; also use the invariants and contracts.

D01 | Count without indexing

Guide: G05, G08

Hint 1

A linked-list cursor already identifies the remaining suffix.

Hint 2

Maintain: counted nodes + nodes in the remaining suffix = original length.

Why this works

Rebinding xs advances a local cursor; it does not change a pair. Each iteration consumes exactly one node.

```
def count_nodes(xs):
    n = 0
    while not is_none(xs):
        n = n + 1
        xs = tail(xs)
    return n
```

Cost: O(n) time; O(1) auxiliary space.

D02 | Alternating checksum

Guide: G06

Hint 1

Carry one piece of state besides the total.

Hint 2

After consuming a value, multiply the sign by -1.

Why this works

The state is the total of the processed prefix and the sign to apply next. Flipping the sign after every node handles negative input naturally.

```
def alternating_total(xs):
    total = 0
    sign = 1
    while not is_none(xs):
        total = total + sign * head(xs)
        sign = -sign
        xs = tail(xs)
    return total
```

Cost: O(n) time; O(1) auxiliary space.

D03 | Adjacent gaps, without a first element

Guide: G06, G08

Hint 1

An output value requires TWO input nodes.

Hint 2

The recursive suffix starts at tail(xs), not tail(tail(xs)).

Why this works

Adjacent pairs overlap: (x0,x1), then (x1,x2). Moving two nodes would skip comparisons. The singleton base case stops before reading a nonexistent second value.

```
def adjacent_gaps(xs):
    if is_none(xs) or is_none(tail(xs)):
        return None
    return pair(head(tail(xs)) - head(xs), adjacent_gaps(tail(xs)))
```

Cost: O(n) time; O(n) fresh output and O(n) recursive stack.

D04 | Prefix totals with an initial balance

Guide: G06

Hint 1

Define the accumulator as the balance BEFORE the current node.

Hint 2

The newly computed balance is both the output value and the next accumulator.

Why this works

Each call receives exactly the summary of earlier input that it needs. It consumes the head once and passes the updated summary to the suffix.

```
def running_totals(xs, start):
    if is_none(xs):
        return None
    updated = start + head(xs)
    return pair(updated, running_totals(tail(xs), updated))
```

Cost: O(n) time; O(n) output and O(n) recursive stack.

D05 | Most expensive leg

Guide: G06, G09

Hint 1

Reuse the adjacent-pair traversal, but change the aggregation.

Hint 2

The matrix access is mat[from_house][to_house].

Why this works

Visit each directed leg once and keep the maximum seen. The empty neutral value 0 is valid because all costs are nonnegative.

```
def route_peak(mat, route):
    best = 0
    while not is_none(route) and (not is_none(tail(route))):
        best = max(best, mat[head(route)][head(tail(route))])
        route = tail(route)
    return best
```

Cost: O(n) route time; O(1) auxiliary space.

D06 | Select by position

Guide: G07, G08

Hint 1

A helper needs the remaining list and the next index.

Hint 2

Compute the selected suffix; prepend the current value only when its position passes.

Why this works

The recursive result preserves the suffix order. The reference evaluates a pure predicate while unwinding; with an effectful predicate, evaluation order would need an explicit contract.

```
def keep_positions(xs, pred):

    def walk(rest, i):
        if is_none(rest):
            return None
        later = walk(tail(rest), i + 1)
        return pair(head(rest), later) if pred(i) else later
    return walk(xs, 0)
```

Cost: O(n) time; O(k) output for k kept values and O(n) stack.

D07 | Compress equal runs

Guide: G08, G17

Hint 1

Finish counting the first run before recursing.

Hint 2

The next recursive input is the first node with a different value, or None.

Why this works

The inner loop consumes one whole run; the outer recursion processes the untouched suffix. Each input node belongs to exactly one counted run.

```
def encode_runs(xs):
    if is_none(xs):
        return None
    value = head(xs)
    count = 0
    rest = xs
    while not is_none(rest) and head(rest) == value:
        count = count + 1
        rest = tail(rest)
    return pair(pair(value, count), encode_runs(rest))
```

Cost: O(n) time; O(r) output and O(r) stack for r runs.

D08 | Expand run records

Guide: G08

Hint 1

There are two kinds of progress: fewer copies left, or fewer records left.

Hint 2

A repeat helper can carry the current value, remaining count and later records.

Why this works

Emit one node per repetition. Only after the count reaches zero do you move to the next record. Output size determines the unavoidable work.

```
def decode_runs(runs):

    def repeat(value, count, rest):
        if count == 0:
            return decode_runs(rest)
        return pair(value, repeat(value, count - 1, rest))
    if is_none(runs):
        return None
    record = head(runs)
    return repeat(head(record), tail(record), tail(runs))
```

Cost: O(N+r) time and O(N+r) stack; O(N) output for N expanded values and r records. With positive counts r <= N.

D09 | Non-overlapping chunk totals

Guide: G06, G08

Hint 1

Consume up to k values using a local cursor.

Hint 2

Recurse on the cursor after the chunk, not on tail of the original input.

Why this works

A bounded inner loop handles one chunk, including a short final chunk. The recursive call starts exactly where that loop stopped.

```
def chunk_sums(xs, k):
    if is_none(xs):
        return None
    total = 0
    used = 0
    while used < k and (not is_none(xs)):
        total = total + head(xs)
        used = used + 1
        xs = tail(xs)
    return pair(total, chunk_sums(xs, k))
```

Cost: O(n) time; O(ceil(n/k)) output and recursive stack.

D10 | Transpose a rectangle

Guide: G04, G09

Hint 1

An input column becomes an output row.

Hint 2

out[c][r] should equal mat[r][c].

Why this works

The output has C rows, each of length R. A new row is created for each column, preventing aliasing between output rows.

```
def transpose(mat):
    rows = len(mat)
    cols = len(mat[0])
    out = [None] * cols
    for c in range(cols):
        out[c] = [0] * rows
        for r in range(rows):
            out[c][r] = mat[r][c]
    return out
```

Cost: O(RC) time and output; O(1) auxiliary space excluding the output.

D11 | Snake across rows

Guide: G09

Hint 1

Separate the count of earlier rows from the position inside this row.

Hint 2

The offset is c on even rows and cols-1-c on odd rows.

Why this works

The block start is r*cols+1. Parity changes only the offset within the current block. This is the row-oriented transfer of Practice A Q4.

```
def snake_rows(rows, cols):
    out = [None] * rows
    for r in range(rows):
        out[r] = [0] * cols
        for c in range(cols):
            offset = c if r % 2 == 0 else cols - 1 - c
            out[r][c] = r * cols + offset + 1
    return out
```

Cost: O(RC) time and output; O(1) auxiliary space.

D12 | Postfix with a unary operator

Guide: G10

Hint 1

Unary and binary operators have different arities: numbers of arguments.

Hint 2

For a binary operator, the first pop is the RIGHT operand.

Why this works

The stack occupies indices 0 through top-1 inside a preallocated list. Reading the right operand at top-1 and the left at top-2 preserves order. A binary operator reduces top by one and replaces the two used values with the result; stale cells beyond top are ignored.

```
def eval_postfix_extended(tokens):
    stack = [None] * len(tokens)
    top = 0
    for i in range(len(tokens)):
        token = tokens[i]
        if token == 'neg':
            stack[top - 1] = -stack[top - 1]
        elif is_string(token):
            right = stack[top - 1]
            left = stack[top - 2]
            top = top - 1
            if token == '+':
                value = left + right
            elif token == '-':
                value = left - right
            elif token == '*':
                value = left * right
            else:
                value = left / right
            stack[top - 1] = value
        else:
            stack[top] = token
            top = top + 1
    return stack[0]
```

Cost: O(t) time; O(t) worst-case stack for t tokens.

D13 | Reverse the links

Guide: G19

Hint 1

Keep the reversed prefix and the unprocessed suffix as two cursors.

Hint 2

Save tail(xs) before redirecting xs to the reversed prefix.

Why this works

Every iteration moves one existing node from the unprocessed suffix to the reversed prefix. Saving nxt prevents losing access to the remaining nodes.

```
def d_reverse(xs):
    previous = None
    while not is_none(xs):
        nxt = tail(xs)
        set_tail(xs, previous)
        previous = xs
        xs = nxt
    return previous
```

Cost: O(n) time; O(1) auxiliary space; zero new pairs.

D14 | Destructive filtering

Guide: G19

Hint 1

You need the first and last kept node.

Hint 2

Save the next input node before changing a kept node's tail.

Why this works

The reference detaches each kept node and appends it to a growing result. It updates last only when keeping a node, so gaps in the original list do not matter.

```
def d_keep(xs, pred):
    result = None
    last = None
    while not is_none(xs):
        nxt = tail(xs)
        if pred(head(xs)):
            set_tail(xs, None)
            if is_none(result):
                result = xs
            else:
                set_tail(last, xs)
            last = xs
        xs = nxt
    return result
```

Cost: O(n) time; O(1) auxiliary space; zero new pairs.

D15 | Cut at a precise boundary

Guide: G19

Hint 1

k=0 needs no link change.

Hint 2

For k>0, stop at node k-1, save its tail, then cut.

Why this works

The cut removes exactly one connection. The output record is not a new linked node and is explicitly permitted by this exercise.

```
def d_split_at(xs, k):
    if k == 0:
        return [None, xs]
    first = xs
    for _ in range(k - 1):
        xs = tail(xs)
    second = tail(xs)
    set_tail(xs, None)
    return [first, second]
```

Cost: O(k) time (O(1) when k=0); O(1) auxiliary space; zero new pairs.

D16 | Merge descending chains

Guide: G20

Hint 1

The next result node is the larger available head.

Hint 2

After choosing a node, attach the merge of the remaining lists to its tail.

Why this works

The direction reverses the comparison in the ascending algorithm. Choosing xs on equality specifies stable cross-input tie behavior.

```
def d_merge_desc(xs, ys):
    if is_none(xs):
        return ys
    if is_none(ys):
        return xs
    if head(xs) >= head(ys):
        set_tail(xs, d_merge_desc(tail(xs), ys))
        return xs
    set_tail(ys, d_merge_desc(xs, tail(ys)))
    return ys
```

Cost: O(n+m) time and O(n+m) worst-case recursive stack; zero new pairs.

D17 | Destructive merge sort

Guide: G20, G21

Hint 1

A slow cursor and a fast cursor locate a midpoint.

Hint 2

Cut before recursive calls; merge the recursively sorted halves.

Why this works

The empty/singleton base prevents splitting forever. Cutting creates independent recursive subproblems. Stable halves plus left-first merge ties preserve original duplicate order.

```
def d_merge_asc(xs, ys):
    if is_none(xs):
        return ys
    if is_none(ys):
        return xs
    if head(xs) <= head(ys):
        set_tail(xs, d_merge_asc(tail(xs), ys))
        return xs
    set_tail(ys, d_merge_asc(xs, tail(ys)))
    return ys

def d_merge_sort(xs):
    if is_none(xs) or is_none(tail(xs)):
        return xs
    slow = xs
    fast = tail(xs)
    while not is_none(fast) and (not is_none(tail(fast))):
        slow = tail(slow)
        fast = tail(tail(fast))
    second = tail(slow)
    set_tail(slow, None)
    return d_merge_asc(d_merge_sort(xs), d_merge_sort(second))
```

Cost: O(n log n) time; O(n) worst-case auxiliary stack with the recursive merge shown; zero new pairs.

D18 | Stable destructive partition

Guide: G19, G20

Hint 1

Build two results, each with a first and last node.

Hint 2

Detach each consumed node, then append it to exactly one result.

Why this works

Save the old tail first. Detaching avoids accidental cross-links between the two partitions. Four endpoint variables suffice without sentinel pairs.

```
def d_partition(xs, pred):
    yes = None
    no = None
    ylast = None
    nlast = None
    while not is_none(xs):
        nxt = tail(xs)
        set_tail(xs, None)
        if pred(head(xs)):
            if is_none(yes):
                yes = xs
            else:
                set_tail(ylast, xs)
            ylast = xs
        else:
            if is_none(no):
                no = xs
            else:
                set_tail(nlast, xs)
            nlast = xs
        xs = nxt
    return [yes, no]
```

Cost: O(n) time; O(1) auxiliary space; zero new pairs.

D19 | Can the robot reach the goal?

Guide: G11-G13

Hint 1

The failed-branch result is False and the goal result is True.

Hint 2

Store the OR of the branches, restore the cell, then return the stored result.

Why this works

Boolean OR can stop exploring after success, but restoration must still execute in the enclosing call. A direct return after marking is unsafe.

```
def reachable(maze, r, c):
    if r < 0 or r >= len(maze) or c < 0 or (c >= len(maze[0])):
        return False
    if maze[r][c] == '#':
        return False
    if maze[r][c] == 'G':
        return True
    old = maze[r][c]
    maze[r][c] = '#'
    result = (
        reachable(maze, r - 1, c) or reachable(maze, r + 1, c) or reachable(maze, r, c -
        1) or reachable(maze, r, c + 1)
    )
    maze[r][c] = old
    return result
```

Cost: Backtracking can take exponential time in V cells; O(V) recursion depth.

D20 | Count simple paths

Guide: G11-G13

Hint 1

Keep the traversal but change how branch results combine.

Hint 2

A blocked branch contributes 0; reaching G contributes 1; add all four children.

Why this works

The path-local mark excludes repetitions only on the current candidate. Restoring allows another path to legitimately use the same cell.

```
def count_simple_paths(maze, r, c):
    if r < 0 or r >= len(maze) or c < 0 or (c >= len(maze[0])):
        return 0
    if maze[r][c] == '#':
        return 0
    if maze[r][c] == 'G':
        return 1
    old = maze[r][c]
    maze[r][c] = '#'
    result = (
        count_simple_paths(maze, r - 1, c) + count_simple_paths(maze, r + 1, c) +
        count_simple_paths(maze, r, c - 1) + count_simple_paths(maze, r, c + 1)
    )
    maze[r][c] = old
    return result
```

Cost: Exponential path enumeration in V; O(V) recursive stack.

D21 | A key changes the state

Guide: G13

Hint 1

The start must be revisited with a key in the example. A single visited bit per cell would forbid that.

Hint 2

Mark the expanded state after applying the effect of the current cell.

Why this works

A path can repeat a coordinate while visiting a new state. Since the key is never consumed, two states per cell suffice. The reference explores simple paths in this expanded graph.

```
def shortest_with_key(maze, r, c):
    rows = len(maze)
    cols = len(maze[0])
    seen = [None] * rows
    for i in range(rows):
        seen[i] = [None] * cols
        for j in range(cols):
            seen[i][j] = [False, False]

    def visit(r, c, has_key):
        if r < 0 or r >= rows or c < 0 or (c >= cols):
            return math_inf
        cell = maze[r][c]
        if cell == '#' or (cell == 'D' and (not has_key)):
            return math_inf
        has_key = has_key or cell == 'K'
        state = 1 if has_key else 0
        if seen[r][c][state]:
            return math_inf
        if cell == 'G':
            return 0
        seen[r][c][state] = True
        best = (
            min(visit(r - 1, c, has_key), visit(r + 1, c, has_key), visit(r, c - 1,
            has_key), visit(r, c + 1, has_key))
        )
        seen[r][c][state] = False
        return 1 + best
    return visit(r, c, False)
```

Cost: Exponential worst-case search over up to 2RC states; O(RC) state marks and stack.

D22 | Pay on entry

Guide: G12-G13

Hint 1

Fewest moves and cheapest path are different objectives.

Hint 2

For each candidate neighbor: its entry cost + its best remaining cost.

Why this works

All entry costs are positive, so removing a cycle strictly reduces cost; some optimal path is simple. That justifies path-local cell marking. The reference saves and restores numeric values.

```
def cheapest_path(costs, r, c, gr, gc):
    rows = len(costs)
    cols = len(costs[0])
    steps = [[-1, 0], [1, 0], [0, -1], [0, 1]]

    def visit(r, c):
        if r < 0 or r >= rows or c < 0 or (c >= cols):
            return math_inf
        if is_none(costs[r][c]):
            return math_inf
        if r == gr and c == gc:
            return 0
        old = costs[r][c]
        costs[r][c] = None
        best = math_inf
        for i in range(4):
            nr = r + steps[i][0]
            nc = c + steps[i][1]
            if nr >= 0 and nr < rows and (nc >= 0) and (nc < cols):
                if not is_none(costs[nr][nc]):
                    best = min(best, costs[nr][nc] + visit(nr, nc))
        costs[r][c] = old
        return best
    return visit(r, c)
```

Cost: Exponential worst-case time; O(RC) stack. This is not a general efficient weighted-search algorithm.

D23 | Break one wall

Guide: G13

Hint 1

A cell reached with the allowance intact differs from the same cell reached after using it.

Hint 2

At a wall, reject if already used; otherwise switch the flag before checking visited state.

Why this works

Expanded state records the resource that determines future moves. A shortest path never needs to repeat the same expanded state, so path-local marks are valid.

```
def shortest_one_break(maze, r, c):
    rows = len(maze)
    cols = len(maze[0])
    seen = [None] * rows
    for i in range(rows):
        seen[i] = [None] * cols
        for j in range(cols):
            seen[i][j] = [False, False]

    def visit(r, c, used):
        if r < 0 or r >= rows or c < 0 or (c >= cols):
            return math_inf
        if maze[r][c] == '#':
            if used:
                return math_inf
            used = True
        state = 1 if used else 0
        if seen[r][c][state]:
            return math_inf
        if maze[r][c] == 'G':
            return 0
        seen[r][c][state] = True
        best = (
            min(visit(r - 1, c, used), visit(r + 1, c, used), visit(r, c - 1, used),
            visit(r, c + 1, used))
        )
        seen[r][c][state] = False
        return 1 + best
    return visit(r, c, False)
```

Cost: Exponential worst-case time over at most 2RC states; O(RC) auxiliary space.

D24 | Exactly k moves, revisits allowed

Guide: G13, G16, G21

Hint 1

The decreasing move budget guarantees termination without visited marks.

Hint 2

Only when remaining==0 do you test whether this walk ends at G.

Why this works

Future choices depend only on position and remaining moves because revisits are legal. That complete state permits sound caching. Earlier arrival at G is not a terminating success.

```
def walks_exactly(maze, r, c, k):
    rows = len(maze)
    cols = len(maze[0])
    cache = [None] * (k + 1)
    for left in range(k + 1):
        cache[left] = [None] * rows
        for i in range(rows):
            cache[left][i] = [None] * cols

    def visit(r, c, left):
        if r < 0 or r >= rows or c < 0 or (c >= cols):
            return 0
        if maze[r][c] == '#':
            return 0
        if left == 0:
            return 1 if maze[r][c] == 'G' else 0
        if is_none(cache[left][r][c]):
            cache[left][r][c] = (
                visit(r - 1, c, left - 1) + visit(r + 1, c, left - 1) + visit(r, c - 1,
                left - 1) + visit(r, c + 1, left - 1)
            )
        return cache[left][r][c]
    return visit(r, c, k)
```

Cost: O((k+1)RC) time and cache; O(k) recursive stack, counting arithmetic as constant-time.

D25 | Lazy strided characters

Guide: G14-G15

Hint 1

Capture the original string once and carry only an index.

Hint 2

The recursive call belongs inside lambda, not inside pair's eagerly evaluated arguments.

Why this works

The first call creates at most one pair and a thunk. Advancing by step occurs only when the tail is forced; no suffix string is copied.

```
def strided_chars(text, start, step):

    def build(i):
        if i >= len(text):
            return None
        return pair(text[i], lambda: build(i + step))
    return build(start)
```

Cost: O(1) construction and work per new node; O(m) total produced nodes for prefix length m.

D26 | Lazy adjacent differences

Guide: G14-G16

Hint 1

One output requires a current source node and its successor.

Hint 2

Retain that successor; the next output starts from it.

Why this works

Lookahead is necessary for adjacent differences. Caching the successor avoids forcing the same source tail twice inside one output step. Output memoization makes repeat access share results.

```
# Include memo_fun from G16.
def stream_gaps(s):
    if is_none(s):
        return None
    rest = stream_tail(s)
    if is_none(rest):
        return None
    return pair(head(rest) - head(s), memo_fun(lambda: stream_gaps(rest)))
```

Cost: O(1) construction and O(m) work for m outputs under the source assumption; O(m) retained output.

D27 | Lazy running balance

Guide: G06, G15-G16

Hint 1

This has the same state invariant as D04.

Hint 2

Both advancing the source and building the next output belong inside the delayed function.

Why this works

The strict-list scan becomes a stream scan by delaying the recursive suffix. A return statement is still needed from the outer function.

```
# Include memo_fun from G16.
def stream_scan(s, total):
    if is_none(s):
        return None
    updated = total + head(s)
    return pair(updated, memo_fun(lambda: stream_scan(stream_tail(s), updated)))
```

Cost: O(1) construction; O(m) work and retained output for m generated elements.

D28 | Fair interleaving

Guide: G15

Hint 1

After emitting the head of s, swap the roles of the two sources.

Hint 2

The delayed next call is interleave(t, stream_tail(s)).

Why this works

Swapping roles guarantees progress from both infinite sources. The empty case returns the surviving stream directly; its existing memoization properties are unchanged.

```
# Include memo_fun from G16.
def interleave(s, t):
    if is_none(s):
        return t
    return pair(head(s), memo_fun(lambda: interleave(t, stream_tail(s))))
```

Cost: O(1) construction; O(m) work for m outputs assuming O(1) source steps.

D29 | Expand a stream of runs

Guide: G15-G17

Hint 1

Carry current value, repetitions remaining, and the source node for the current run.

Hint 2

Force the next run only after the last output of the current run is consumed.

Why this works

The source cursor stays on the same run while its copies are emitted. A delayed boundary step advances the source exactly once.

```
# Include memo_fun from G16.
def expand_run_stream(s):

    def emit(value, left, source):

        def later():
            if left > 1:
                return emit(value, left - 1, source)
            return expand_run_stream(stream_tail(source))
        return pair(value, memo_fun(later))
    if is_none(s):
        return None
    record = head(s)
    return emit(head(record), tail(record), s)
```

Cost: O(1) construction and work per output; O(m) retained output for m elements.

D30 | Self-referential smooth numbers

Guide: G16-G18

Hint 1

An equality case in stream merge must advance BOTH sources.

Hint 2

Create the head 1 before any expression using S is forced.

Why this works

The seed makes the circular definition productive: every later value depends on smaller, already available values. Memoization prevents repeated reconstruction of shared prefixes. Removing duplicates requires consuming equal candidates together.

```
# Include memo_fun from G16.
def smooth_numbers():

    def scale(s, factor):
        return pair(head(s) * factor, memo_fun(lambda: scale(stream_tail(s), factor)))

    def merge(s, t):
        a = head(s)
        b = head(t)
        if a < b:
            return pair(a, memo_fun(lambda: merge(stream_tail(s), t)))
        if b < a:
            return pair(b, memo_fun(lambda: merge(s, stream_tail(t))))
        return pair(a, memo_fun(lambda: merge(stream_tail(s), stream_tail(t))))
    s = pair(1, memo_fun(lambda: merge(scale(s, 2), merge(scale(s, 3), scale(s, 5)))))
    return s
```

Cost: O(n) stream-node work for n outputs with fixed three multipliers and unit-cost arithmetic; O(n) retained nodes. Big integers add arithmetic cost.

D31 | Find a boundary, not just a match

Guide: G21

Hint 1

The answer is a boundary in [0,n], not necessarily an existing element.

Hint 2

Maintain all indices before lo are too small and all indices at or after hi meet the threshold.

Why this works

A half-open interval [lo,hi) avoids special cases for an empty list. Equality moves hi left, so repeated matches cannot hide an earlier valid position.

```
def lower_bound(a, target):
    lo = 0
    hi = len(a)
    while lo < hi:
        mid = (lo + hi) // 2
        if a[mid] < target:
            lo = mid + 1
        else:
            hi = mid
    return lo
```

Cost: O(log(n+1)) time; O(1) auxiliary space.

D32 | Preserve a nested shape

Guide: G22

Hint 1

There are three representation cases: empty, atomic leaf, and pair.

Hint 2

In the pair case, recursively transform BOTH head and tail.

Why this works

Recursing only on tails would miss nested lists. Testing emptiness first prevents applying f to None. Fresh pairs preserve shape without sharing mutable structural nodes.

```
def tree_map(tree, f):
    if is_none(tree):
        return None
    if not is_pair(tree):
        return f(tree)
    return pair(tree_map(head(tree), f), tree_map(tail(tree), f))
```

Cost: O(N) time and output for N representation nodes/leaves; O(H) stack where H is maximum structural recursion depth.

D33 | Pass the answer onward

Guide: G07, G22

Hint 1

k describes what to do with the completed answer.

Hint 2

The next continuation receives the suffix sum and passes current squared value + suffix sum to the old k.

Why this works

The pending addition is packaged as a function. All recursive calls and continuation applications are in tail position, so Python §3 can reuse control space. However, the chain of n pending closures still occupies O(n) memory. Return the final continuation result.

```
def sum_squares_cps(xs, k):
    if is_none(xs):
        return k(0)
    value = head(xs)
    return sum_squares_cps(tail(xs), lambda result: k(value * value + result))
```

Cost: O(n) time and O(n) pending closures; O(1) control-stack depth under Python §3 proper tail calls, excluding work done by the supplied final continuation.

D34 | A closure with undo

Guide: G04, G07, G16

Hint 1

Keep a linked history whose head is the current value.

Hint 2

Add by prepending a pair; undo by moving to its tail, unless only the initial value remains.

Why this works

Each factory call creates an independent history chain. The inner function declares nonlocal history because it rebinds that enclosing name. Undo drops the most recent history node, while the singleton base prevents undoing past start.

```
def make_undo_counter(start):
    history = pair(start, None)

    def counter(command, amount):
        nonlocal history
        if command == 'add':
            history = pair(head(history) + amount, history)
        elif command == 'undo' and (not is_none(tail(history))):
            history = tail(history)
        return head(history)
    return counter
```

Cost: O(1) construction and per command; O(a) reachable history after a un-undone additions.

D35 | Evaluate lexical let expressions

Guide: G23

Hint 1

Separate syntax selection, variable lookup, and environment extension.

Hint 2

A binding record is pair(name,value); a new environment is pair(binding,old_env).

Why this works

Lookup scans newest bindings first. The right-hand side must be evaluated before prepending the new binding; otherwise shadowing can change its meaning. This is a small evaluator, not a full implementation of the course evaluator.

```
def eval_let(expr, env):
    tag = expr[0]
    if tag == 'num':
        return expr[1]
    if tag == 'var':
        rest = env
        while not is_none(rest):
            binding = head(rest)
            if head(binding) == expr[1]:
                return tail(binding)
            rest = tail(rest)
        return error('Unbound variable')
    if tag == 'add':
        return eval_let(expr[1], env) + eval_let(expr[2], env)
    if tag == 'mul':
        return eval_let(expr[1], env) * eval_let(expr[2], env)
    value = eval_let(expr[2], env)
    extended = pair(pair(expr[1], value), env)
    return eval_let(expr[3], extended)
```

Cost: O(E*D) worst case for E syntax nodes and maximum environment depth D; O(E+D) upper bound on allocated bindings and stack.

D36 | Decode a route and stream its cost

Guide: G06, G09, G15-G16

Hint 1

Two summaries of the past are needed: current house and total cost.

Hint 2

Decode destination first, then read mat[current][destination], then update both state variables.

Why this works

This composes adjacent directed costs, prefix decoding and delayed scans without an intermediate route. The helper state gives the meaning of every output and prevents swapping matrix coordinates.

```
# Include memo_fun from G16.
def route_cost_stream(mat, start, deltas):

    def build(source, current, total):
        if is_none(source):
            return None
        dest = current + head(source)
        updated = total + mat[current][dest]
        return (
            pair(updated, memo_fun(lambda: build(stream_tail(source), dest, updated)))
        )
    return build(deltas, start, 0)
```

Cost: O(1) construction; O(n) prefix time and retained output for n moves, excluding the existing matrix/source.